

使用 Actions SDK (第 1 級) 為 Google 助理建構動作

來源：<https://codelabs.developers.google.com/codelabs/actions-sdk-1?hl=zh-tw>

步驟數：7

瞭解使用 Google 助理開發應用程式的基本概念。

目錄

1. [1. 總覽](#)
2. [2. 設定](#)
3. [3. 發起對話](#)
4. [4. 建立動作的對話](#)
5. [5. 實作執行要求](#)
6. [6. 使用動作建構工具視覺化呈現動作](#)
7. [7. 恭喜！](#)

1. 總覽

警告：對話動作已於 2023 年 6 月 13 日淘汰。詳情請參閱 [Conversational Actions 終止支援](#)。

Google 助理開發人員平台可讓您建立軟體，在超過 10 億部裝置上擴充 Google 助理 (虛擬個人助理) 的功能，包括智慧音箱、手機、汽車、電視、耳機等。使用者可以與 Google 助理對話，完成購物或預約車輛等事項。開發人員可以透過 Google 助理開發人員平台，輕鬆建立及管理使用者與第三方服務之間，令人愉悅且有效的對話體驗。

本程式碼研究室涵蓋使用 Google 助理 Actions SDK 進行開發的初學者概念，您不必具備任何平台經驗，即可完成本課程。在本程式碼研究室中，您將建構簡單的 Google 助理動作，在使用者開始 Gryffinberg 神話國度的冒險時，告訴他們運勢。在 Actions SDK 第 2 級程式碼研究室中，您會進一步建構這項動作，根據使用者的輸入內容自訂運勢。

建構項目

在本程式碼研究室中，您將建構簡單的動作，並具備下列功能：

- 向使用者傳送問候訊息
- 向使用者提出問題，並根據使用者的選擇做出適當回應
- 提供建議方塊，使用者點選即可輸入內容
- 根據使用者是否為回訪者，修改對使用者的問候訊息

完成本程式碼研究室後，完成的動作會採用下列對話流程 (麥克風旁的文字代表使用者輸入內容，揚聲器旁的文字則代表動作的回應)：

talk to my test app  

 Here's the test version of my test app.
A wondrous greeting, adventurer! Welcome to the mythical land of Gryffinberg! Based on your clothes, you are not from around these lands. It looks like you're on your way to an epic journey. Before you continue on your quest, would you like your fortune told?

yes  

 Your future depends on the item you choose to use for your quest. Choose wisely! Farewell, stranger.

'my test app' left the conversation

talk to my test app  



Getting the test version of my test app.
A wondrous greeting, adventurer! Welcome to the mythical land of Gryffinberg! Based on your clothes, you are not from around these lands. It looks like you're on your way to an epic journey. Before you continue on your quest, would you like your fortune told?

no thank you  



I understand, stranger. Best of luck on your quest! Farewell.

'my test app' left the conversation

課程內容

- 如何在 Actions 控制台中建立專案
- 如何使用 gactions 工具，在 Actions 控制台和本機檔案系統之間推送及提取動作專案
- 如何在使用者叫用動作後傳送提示
- 如何處理使用者輸入內容並傳回回應
- 如何在 Actions 模擬工具中測試動作
- 如何使用 Cloud Functions 編輯器實作執行要求

軟硬體需求

環境中必須有下列工具：

- 您選擇的 IDE 或文字編輯器
- 終端機，用於執行 [Node.js](#) 和 [npm](#) 的殼層指令
- 網路瀏覽器，例如 [Google Chrome](#)

如果您偏好使用更多視覺化工具建構動作，請改為完成「[使用 Actions Builder 建構 Google 助理動作 \(第 1 級\)](#)」程式碼研究室，該研究室會使用主控台的 Actions Builder IDE 建構相同的動作。

步驟 2 · 約 5 分鐘

2. 設定

以下各節說明如何設定開發環境及建立 Actions 專案。

檢查 Google 權限設定

若要測試在本程式碼研究室中建構的動作，您必須啟用必要權限，模擬器才能存取動作。

如要啟用權限，請按照下列步驟操作：

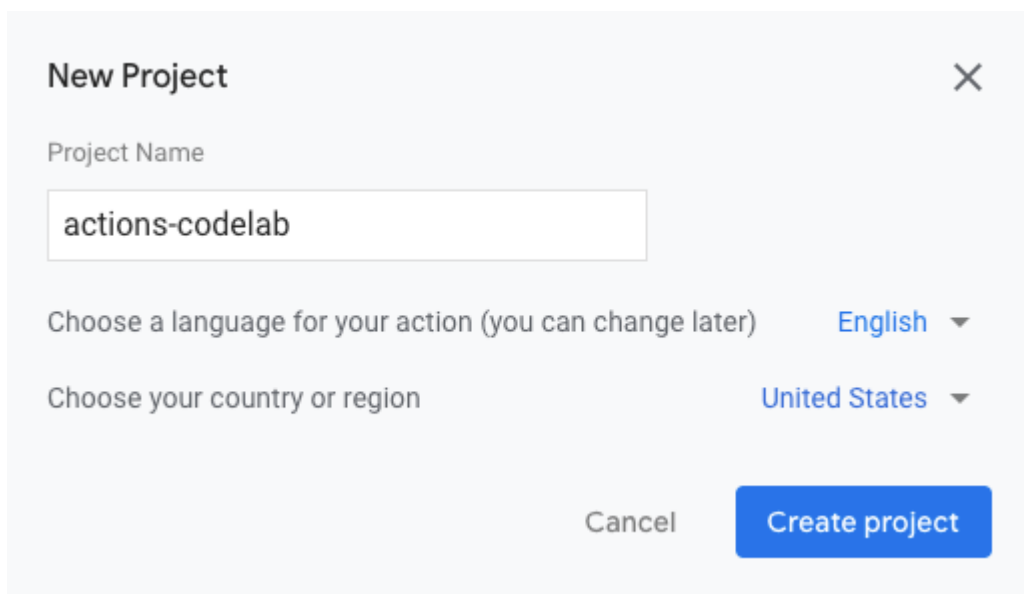
1. 前往「[活動控制項](#)」。
2. 如果尚未登入 Google 帳戶，請先登入。
3. 啟用下列權限：
 - **網路和應用程式活動**
 - 在「網路和應用程式活動」下方，勾選「包括 Chrome 歷史記錄以及採用 Google 服務的網站、應用程式和裝置中的活動記錄」核取方塊。

建立 Actions 專案

動作專案是動作的容器。

如要為本程式碼研究室建立 Actions 專案，請按照下列步驟操作：

1. 開啟 [Actions 管理中心](#)。
2. 按一下 [新專案]。
3. 輸入**專案名稱**，例如 `actions-codelab`。(這個名稱僅供內部參考，稍後，您可以為專案設定外部名稱)。



New Project

Project Name

actions-codelab

Choose a language for your action (you can change later) English

Choose your country or region United States

Cancel Create project

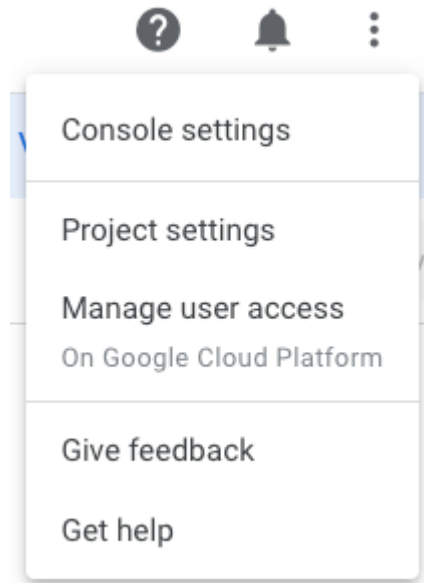
4. 按一下 [Create Project]。
5. 在「您要建構哪種動作？」畫面中，選取「自訂」資訊卡。
6. 點選 [下一步]。
7. 在「你想要如何建構？」畫面中，選取「空白專案」資訊卡。
8. 按一下「開始建構」。

儲存動作的專案 ID

專案 ID 是動作的專屬 ID。本程式碼研究室的幾個步驟會用到專案 ID。

如要擷取專案 ID，請按照下列步驟操作：

1. 在 Actions 控制台中，按一下三個垂直圓點 (在此插入圖示)。
2. 按一下「專案設定」。



3. 複製專案 ID。

建立帳單帳戶關聯

如果您還沒有帳單帳戶，請建立帳單帳戶並與 Google Cloud 中的專案建立關聯，以便稍後使用 Cloud Functions 部署服務。

如要將帳單帳戶與專案建立關聯，請按照下列步驟操作：

1. 前往 [Google Cloud Platform 帳單頁面](#)。
2. 按一下「新增帳單帳戶」或「建立帳戶」。
3. 輸入您的付款資訊。
4. 按一下「開始免費試用」或「提交資訊並啟用計費功能」。
5. 前往 [Google Cloud Platform 帳單頁面](#)。
6. 按一下「我的專案」分頁標籤。
7. 在 Codelab 的 Actions 專案旁，點選「動作」下方的三點圖示。
8. 按一下「變更帳單」。
9. 在下拉式選單中，選取您設定的帳單帳戶。
10. 按一下 [設定帳戶]。

如要避免產生費用，請按照本程式碼研究室結尾「清除專案」一節中的步驟操作。

安裝 gactions 指令列介面

在本程式碼研究室中，您將使用 *gactions* 指令列介面 (CLI) 工具，在 Actions 控制台和本機檔案系統之間同步處理 Actions 專案。

重要字詞：

- **gactions CLI**：這項工具可讓您初始化 Actions 專案檔案、將更新推送至 Actions 控制台，以及部署 Actions 專案等。

如要安裝 gactions CLI，請按照「[安裝 gactions 指令列工具](#)」中的操作說明進行。

下載 Actions 專案

從 Actions 主控台下載 Actions 專案，開始開發動作。

如要下載 Actions 專案，請按照下列步驟操作：

1. 如要建立新目錄並切換至該目錄，請執行下列指令：

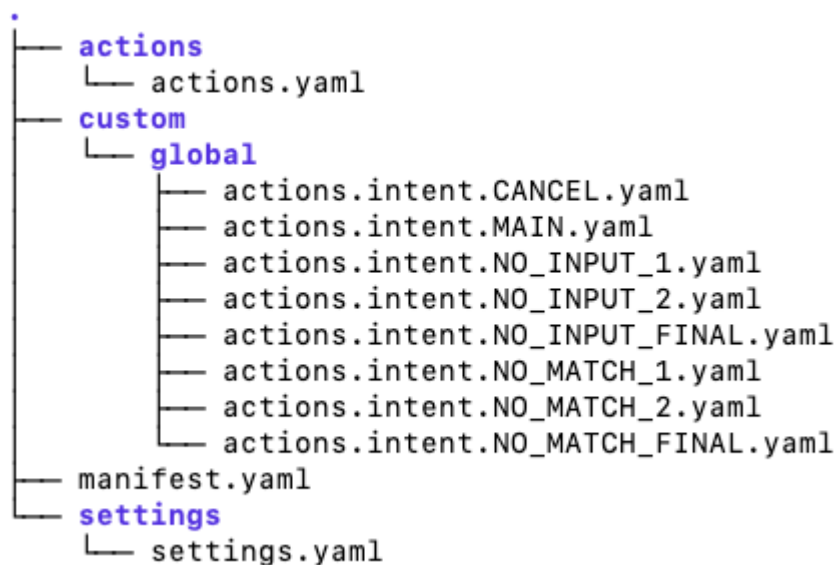
```
mkdir myproject
cd myproject
```

2. 如要將 Actions 專案的設定複製到本機檔案系統，請執行下列指令：

```
gactions pull --project-id <projectID>
```

瞭解檔案結構

從 Actions 管理中心下載的 Actions 專案會以 **YAML** 檔案結構表示。下圖為檔案結構的概略表示法：



檔案結構包含下列項目：

- **actions/**：代表您的 Actions 專案。系統會在叫用動作時呼叫 `actions.yaml`，然後呼叫 `custom/global/actions.intent.MAIN.yaml` 檔案。
- **custom/**：您要用來修改動作的目錄。
- **global/**：這個目錄包含平台自動新增至專案的**系統意圖**。您將在本程式碼研究室稍後的部分，進一步瞭解系統意圖。

- `manifest.yaml`：包含可轉移的資訊，或不屬於任何特定專案，可在專案之間移動。
 - `settings/`：代表 Actions 專案的設定，例如顯示名稱、預設語言代碼和類別。
-

3. 發起對話

使用者會透過**呼叫**與動作展開對話。舉例來說，如果動作名稱為「MovieTime」，使用者可以說出「Ok Google，與 MovieTime 對話」這類句子來叫用動作，其中「MovieTime」是**顯示名稱**。如要將動作部署至正式環境，必須提供顯示名稱；但如要測試動作，則不需要定義顯示名稱。請改用模擬器中的「Talk to my test app」（與我的測試應用程式對話）片語，叫用動作。本節稍後會進一步說明模擬器。

您必須編輯**主要叫用**，定義使用者叫用動作後會發生的情況。

重要字詞：

- **主要叫用**：使用者與動作展開對話的進入點。當使用者說出類似「Ok Google，與『<顯示名稱>』對話」的片語時，主要叫用會觸發並回應使用者。

根據預設，觸發叫用時，動作會提供一般**提示**（「Start building your Action by defining main invocation.」）。

重要字詞：

- **提示**：定義 Action 要傳送給使用者的回覆內容。提示可以是簡單的文字和語音回覆，也可以是資訊卡和圖片等豐富訊息。

您可以使用特殊提示 `Canvas`，向使用者傳回全螢幕圖像。如要進一步瞭解 `Canvas`，請參閱[互動式畫布說明文件](#)。

在下一節中，您將在 `custom/global/actions.intent.MAIN.yaml` 檔案中自訂主要呼叫的提示。

設定主要叫用

您可以在 `actions.intent.MAIN.yaml` 檔案中編輯主要呼叫提示。

如要修改使用者叫用動作時，動作傳回給使用者的提示，請按照下列步驟操作：

1. 在文字編輯器中開啟 `custom/global/actions.intent.MAIN.yaml`。
2. 將 `speech` 欄位（Start building your action...）中的文字替換為下列歡迎訊息：`A wondrous greeting, adventurer! Welcome to the mythical land of Gryffinberg! Based on your clothes, you are not from around these lands. It looks like you're on your way to an epic journey.`

actions.intent.MAIN.yaml

```
handler:
  staticPrompt:
    candidates:
      - promptResponse:
          firstSimple:
            variants:
              - speech: A wondrous greeting, adventurer! Welcome to the mythical land of
                Gryffinberg! Based on your clothes, you are not from around these lands.
                It looks like you're on your way to an epic journey.
  transitionToScene: actions.scene.END_CONVERSATION
```

3. 儲存檔案。

在模擬工具中測試主要叫用

Actions 管理中心提供名為「模擬器」的網頁工具，可供測試動作。這個介面會模擬硬體裝置及其設定，因此您可以與動作對話，就像動作在智慧螢幕、手機、音箱或 KaiOS 上執行一樣。

如要瞭解如何使用 Actions 管理中心模擬器，或在執行下列步驟時遇到任何問題，請參閱「[模擬器](#)」頁面。

現在叫用動作時，系統應會以您新增的自訂提示回應（「冒險家，歡迎來到奇幻世界！...」）。

您可以使用 `gactions deploy preview` 指令在控制台中測試動作，不必更新動作專案的版本。執行這項指令時，您在本機檔案系統中所做的任何變更，都不會傳播至已部署的 Actions 專案版本，但您可以在預覽版本中測試這些變更。

如要在模擬器中測試動作的主要叫用方式，請按照下列步驟操作：

1. 如要將專案部署至 Actions 控制台進行測試，請在終端機中執行下列指令：

```
gactions deploy preview
```

執行這項指令時，系統可能會通知您驗證問題。只要檔案已推送，即可忽略這則通知。

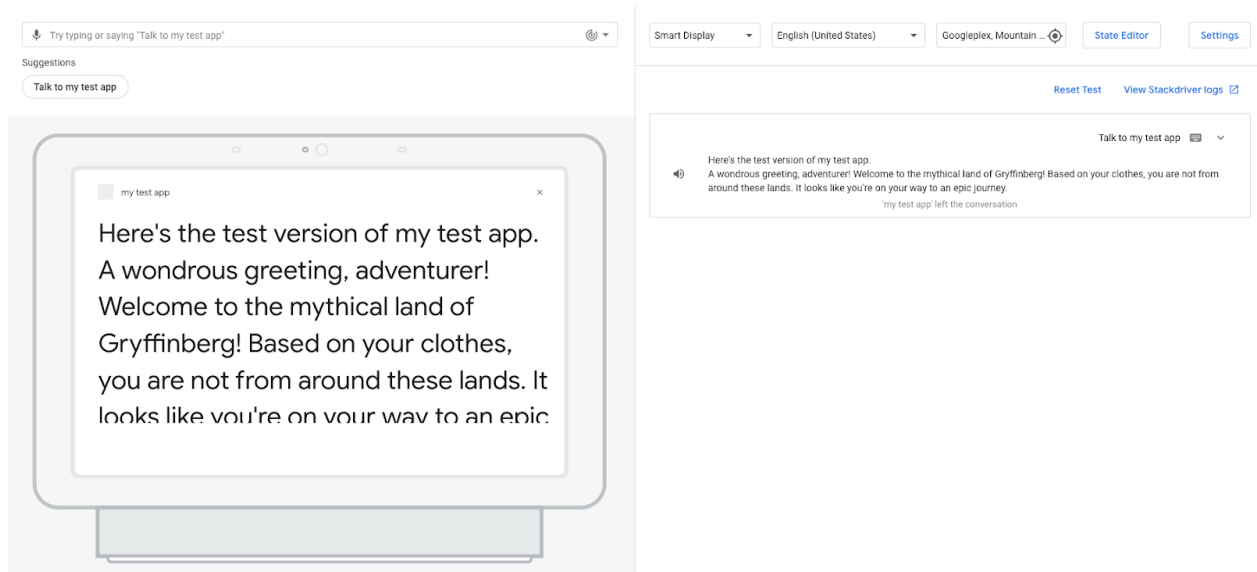
輸出內容應如下所示：

```
✓ Done. You can now test your changes in Simulator with this URL:
http://console.actions.google.com/project/{project-id}/simulator?disableAutoPreview
```

2. 複製提供的網址並貼到瀏覽器中。
3. 如要在模擬工具中叫用動作，請在「Input」（輸入）欄位中輸入 `Talk to my test app`，然後按 `Enter`。

提示：你也可以點選「建議」下方的「與測試版應用程式對話」建議方塊，系統會以處理口語或輸入文字的方式處理這項操作。

你也可以點選麥克風圖示，然後說出「Talk to my test app」來叫用動作。

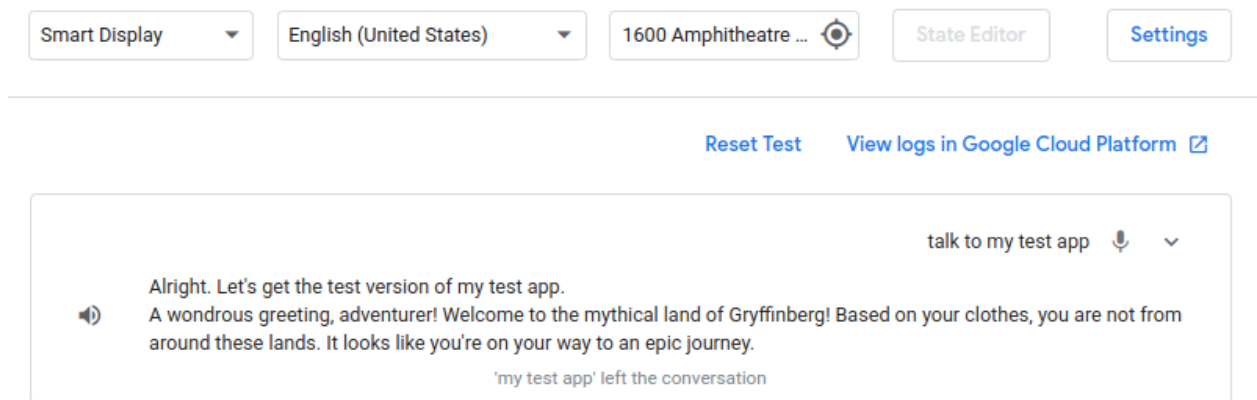


觸發動作的主要叫用時，Google 助理會以自訂的歡迎訊息回應。此時，Google 助理會回覆問候語，然後結束對話。在下一節中，您將修改動作，讓對話繼續進行。

查看事件記錄

在 Actions 控制台的「測試」分頁中，面板會顯示 **事件記錄**，以事件記錄的形式顯示對話記錄。每個事件記錄都會顯示該輪對話期間發生的事件。

你的動作目前有一個事件記錄，其中會顯示使用者的輸入內容（「Talk to my test app」）和動作的回應。下圖顯示動作的事件記錄：



如果點選事件記錄中 **Talk to my test app** 旁的向下箭頭，即可查看該輪對話中發生的事件，並依時間順序排列：

- **userInput**：對應使用者的輸入內容（「與我的測試應用程式對話」）
- **interactionMatch**：對應於使用者輸入內容觸發的 Action 主要叫用回應。按一下箭頭展開這個資料列，即可查看您為主要呼叫新增的提示（A wondrous greeting, adventurer!...）。
- **endConversation**：對應 **Main invocation** 意圖中選取的轉移，目前會結束對話。您將在本程式碼研究室的下一節中，進一步瞭解轉場效果。

事件記錄可讓您瞭解動作的運作方式，如果遇到任何問題，這項工具也可用於偵錯動作。如要查看事件詳細資料，請按一下事件名稱旁的箭頭，如下列螢幕截圖所示：



Here's the test version of my test app.

A wondrous greeting, adventurer! Welcome to the mythical land of Gryffinberg! Based on your clothes, you are not from around these lands. It looks like you're on your way to an epic journey.

'my test app' left the conversation

Event

Scene

▶ userInput actions.scene.START_CONVERSATION

▼ interactionMatch actions.scene.START_CONVERSATION



```
{
  "interactionMatch": {
    "intentId": "actions.intent.MAIN",
    "nextSceneId": "actions.scene.END_CONVERSATION"
  },
  "responses": [
    {
      "firstSimple": {
        "speech": "A wondrous greeting, adventurer! Welcome to the mythical land of Gryffinberg! Based on your clothes, you are not from around these lands. It looks like you're on your way to an epic journey.",
        "text": "A wondrous greeting, adventurer! Welcome to the mythical land of Gryffinberg! Based on your clothes, you are not from around these lands. It looks like you're on your way to an epic journey."
      }
    }
  ]
}
```

Session storage

```
{}
```

User storage

```
{}
```

▶ endConversation actions.scene.END_CONVERSATION

4. 建立動作的對話

定義使用者叫用動作後會發生的情況後，您就可以建構動作對話的其餘部分。繼續進行本程式碼研究室之前，請先熟悉下列用語，瞭解動作的對話運作方式：

重要字詞：

- **場景**：Action 的建構區塊，可處理使用者輸入內容，並通常會產生使用者回應。*自訂意圖*通常會啟動其他場景，但場景也可以透過其他方式啟動。
- **自訂意圖**：定義動作可解讀的使用者輸入內容。您要為這些意圖定義「訓練詞組」。使用者說出的內容與訓練詞組相符或相似時，系統就會視為符合對應意圖。
- **訓練詞組**：可與意圖配對的使用者話語內容範例。Google 助理的自然語言理解 (NLU) 引擎會根據這些訓練詞組，自然延伸出其他類似的詞組。
- **轉場**：定義條件符合時要轉場至哪個場景。

動作可以有一個或多個場景，您必須先啟用每個場景，才能執行動作。您在本程式碼研究室中建構的動作只有一個名為「`Start`」的場景。啟用場景最常見的方式是設定動作，讓使用者輸入內容符合場景中的意圖時，該意圖會觸發轉換至另一個場景並啟用。

舉例來說，假設有一個虛構的動作，可向使用者提供動物相關知識。使用者叫用這項動作時，系統會比對 `Main invocation` 意圖，並觸發轉換至名為「`Facts.`」的場景。這項轉換會啟動 `Facts` 場景，並向使用者傳送下列提示：
「Would you like to hear a fact about cats or dogs?」。 `Facts` 場景內有名為「`Cat`」的自訂意圖，其中包含使用者可能會說的訓練片語，例如「我想聽貓咪小知識」或「貓」。使用者要求聽貓咪冷知識時，系統會比對 `Cat` 意圖，並觸發轉換至名為 `Cat fact.` 的場景。`Cat fact` 場景會啟動並傳送提示給使用者，其中包含貓咪冷知識。

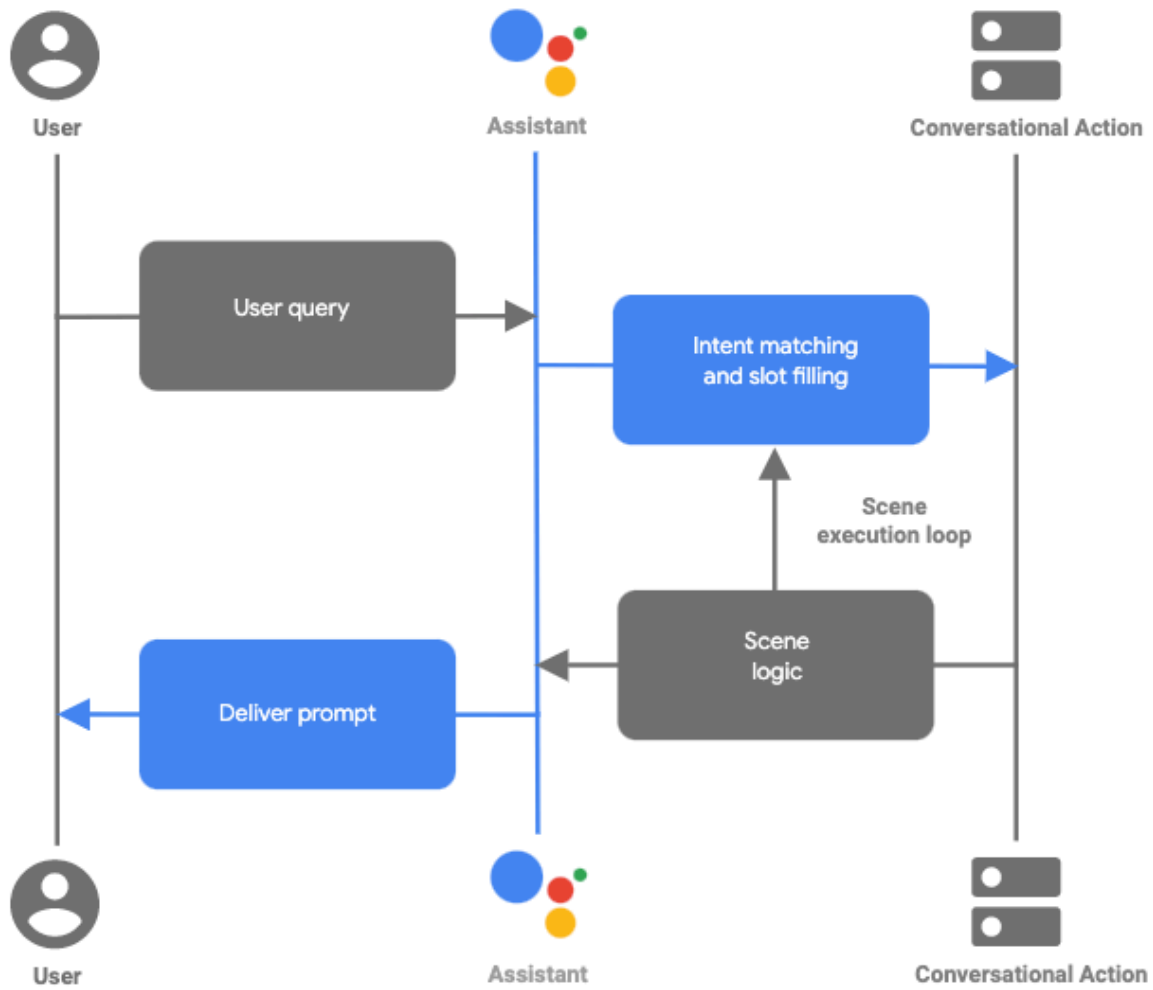


圖 1. 使用 Actions SDK 建構的動作中，一般對話回合的流程。

場景、意圖和轉場效果共同構成對話邏輯，並定義使用者在動作對話中可採取的各種路徑。在下一節中，您將建立場景，並定義使用者叫用動作後，該場景的啟動方式。

從主要叫用項目轉換至場景

在本節中，您將建立名為 `start` 的新場景，向使用者傳送提示，詢問是否要算命。您也會從主要呼叫新增至新的 `start` 場景。

如要建立這個場景並加入轉場效果，請按照下列步驟操作：

1. 在文字編輯器中開啟 `custom/global/actions.intent.MAIN.yaml`。
2. 將 `transitionToScene` 欄位 (`actions.scene.END_CONVERSATION`) 中的文字替換為下列內容：`transitionToScene: start`

`actions.intent.MAIN.yaml`

```
handler:
  staticPrompt:
    candidates:
      - promptResponse:
          firstSimple:
            variants:
              - speech: Welcome to the mythical land of Gryffinberg! Based on your clothes,
                you are not from around these lands. It looks like you're on your way
                to an epic journey.
  transitionToScene: Start
```

這會告知動作從主要叫用轉換至 `Start` 場景。

3. 儲存檔案。

4. 在終端機中，於 `custom` 目錄中建立新的 `scenes` 目錄：

```
mkdir custom/scenes
```

5. 在 `scenes` 目錄中建立名為 `Start.yaml` 的新檔案，代表 Action 中的 `start` 場景：

```
touch custom/scenes/Start.yaml
```

6. 在文字編輯器中開啟 `Start.yaml`。

7. 將下列程式碼貼入 `Start.yaml` 檔案：

Start.yaml

```
onEnter:
  staticPrompt:
    candidates:
      - promptResponse:
          firstSimple:
            variants:
              - speech: Before you continue on your quest, would you like your fortune
                told?
```

在 `Start.yaml` 檔案程式碼中，有一個名為 `onEnter` 的欄位，這是場景生命週期中執行的第一個階段。

關鍵字詞：

- **OnEnter**：場景啟動時執行的階段。 `OnEnter` 定義使用者進入場景時會發生的情況。

在本例中，當使用者首次進入 `Start` 場景時，系統會將提示 (`Before you continue on your quest...`) 新增至提示佇列。

關鍵字詞：

- **提示佇列：**提示會附加到提示佇列中。在使用者回覆之前，Google 助理會向使用者顯示提示佇列中的所有提示。

新增建議方塊

建議晶片會向使用者提供可點選的建議，供動作處理為使用者輸入內容。在本節中，您會新增 **Yes** 和 **No** 建議晶片，這些晶片會顯示在您剛設定的提示下方 (Before you continue on your quest, would you like your fortune told?)，為螢幕裝置上的使用者提供支援。

如要將建議方塊新增至 **Start** 場景的提示，請按照下列步驟操作：

1. 更新 **Start.yaml** 中的程式碼，使其符合下列程式碼片段，其中包含設定建議方塊的程式碼：

Start.yaml

```
onEnter:
  staticPrompt:
    candidates:
      - promptResponse:
          firstSimple:
            variants:
              - speech: Before you continue on your quest, would you like your fortune
                told?
            suggestions:
              - title: "Yes"
              - title: "No"
```

2. 儲存檔案。

在模擬工具中測試動作

此時，動作應從主要叫用轉換至 **Start** 場景，並詢問使用者是否要算命。模擬畫面中也應會顯示建議方塊。

如要在模擬器中測試動作，請按照下列步驟操作：

1. 在終端機執行下列指令：

```
gactions deploy preview
```

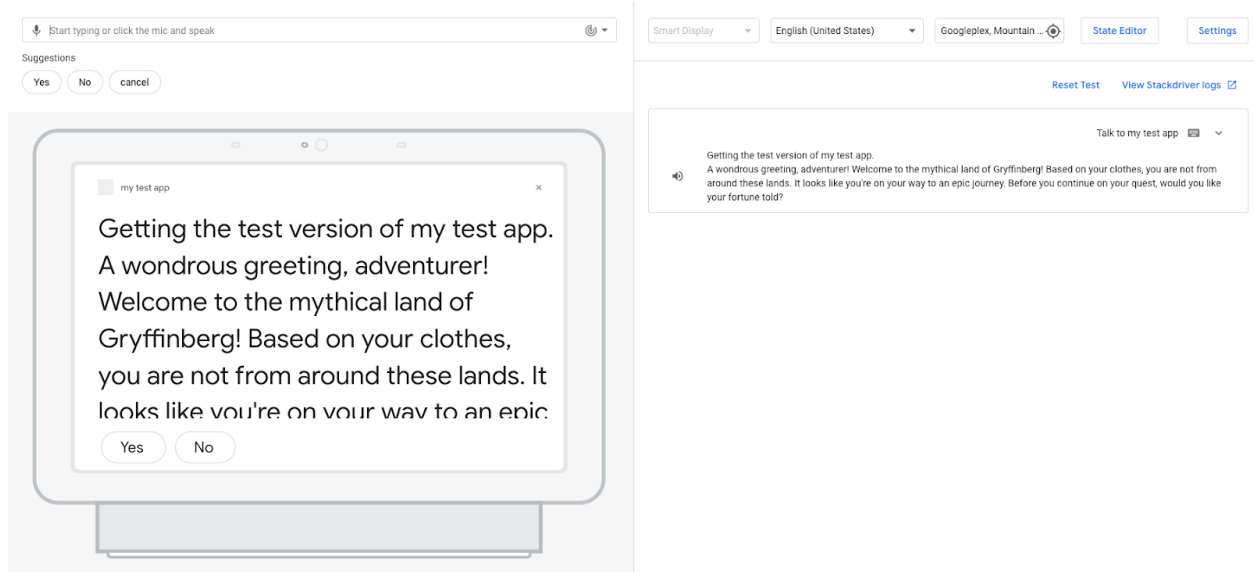
輸出內容應如下所示：

```
✓ Done. You can now test your changes in Simulator with this URL:
http://console.actions.google.com/project/{project-id}/simulator?disableAutoPreview
```

2. 複製提供的網址並貼到瀏覽器中。
3. 按一下「測試」即可前往模擬器。

4. 在「Input」欄位中輸入 `Talk to my test app`，然後按下 `Enter` 鍵。你的動作應回應 `Main invocation` 提示和新增的 `start` 場景提示「在繼續冒險之前，要不要先算算命？」，並顯示建議晶片。

以下螢幕截圖顯示這項互動：



事件記錄會記錄目前工作階段中模擬器的每次互動，因此本程式碼研究室稍早測試的事件記錄會顯示為第一個事件記錄。如要清除事件記錄，請按一下「重設測試」。

5. 按一下「是」或「否」建議晶片，回覆提示。(你也可以說「是」或「否」，或在「輸入」欄位中輸入 `Yes` 或 `No`。)

當你回覆提示時，動作會顯示訊息，指出無法理解你的輸入內容：「很抱歉，我沒聽清楚。可以請你再試一次嗎？」由於您尚未設定 Action，讓 Action 瞭解並回覆「是」或「否」輸入內容，因此 Action 會將您的輸入內容與 `NO_MATCH` 意圖相符。

關鍵字詞：

- `NO_MATCH`：如果使用者說的內容無法與動作中的其他意圖相符，就會比對出這個意圖。
- 系統意圖**：系統預設設定的意圖，可處理常見的系統事件，例如使用者未提供輸入內容。`NO_MATCH` 和與主要叫用項目相關的意圖，都是系統意圖的例子。

根據預設，`NO_MATCH` 系統意圖會提供一般回應，但您可以自訂這些回應，向使用者表示您不瞭解他們輸入的內容。如果 Google 助理無法比對使用者輸入內容，就會在三次後結束與 Action 的對話。

新增 `yes` 和 `no` 意圖

現在使用者可以回應動作提出的問題，因此您可以設定動作來瞭解使用者的回應（「是」或「否」）。在接下來的章節中，您將建立自訂意圖，當使用者說「是」或「否」時，系統就會比對這些意圖，並將這些意圖新增至 `start` 場景。

建立 `yes` 意圖

如要建立 `yes` 意圖，請按照下列步驟操作：

- 在終端機中，於 `custom` 目錄中建立名為 `intents` 的新目錄：

```
mkdir custom/intents
```

2. 在 `intents` 目錄中建立名為 `yes.yaml` 的新檔案：

```
touch custom/intents/yes.yaml
```

3. 在文字編輯器中開啟 `yes.yaml` 。

4. 將包含訓練詞組的下列程式碼片段貼到 `yes.yaml`：

yes.yaml

```
trainingPhrases:
- of course
- let's do it
- ok
- sure
- "y"
- "yes"
```

5. 儲存檔案。

將 `yes` 意圖新增至 `Start` 場景

現在，動作可以瞭解使用者何時表達「是」的意圖。由於使用者正在回應 `Start` 提示 (「Before you continue on your quest, would you like your fortune told?」(繼續冒險前，要不要算算命?))，因此您可以將 `yes` 自訂意圖新增至 `Start` 場景。

如要將這個自訂意圖新增至 `Start` 場景，請按照下列步驟操作：

1. 在文字編輯器中開啟 `custom/scenes/Start.yaml` 。
2. 在 `Start.yaml` 檔案結尾新增 `intentEvents` 和 `yes` 處理常式：

Start.yaml

```
intentEvents:
- handler:
    staticPrompt:
      candidates:
      - promptResponse:
          firstSimple:
            variants:
              - speech: Your future depends on the item you choose to use for your quest.
                Choose wisely! Farewell, stranger.
            intent: "yes"
          transitionToScene: actions.scene.END_CONVERSATION
```

當系統比對出 `yes` 意圖時，提示佇列會加入「你的未來取決於你選擇用於任務的物品...」提示。 `Start` 場景隨即轉換為 `actions.scene.END_CONVERSATION` 系統場景，傳送提示佇列中的提示並結束對話。

在模擬工具中測試 `yes` 意圖

此時，Action 會瞭解使用者何時想聽取運勢，並傳回適當的回應。

如要在模擬器中測試這項意圖，請按照下列步驟操作：

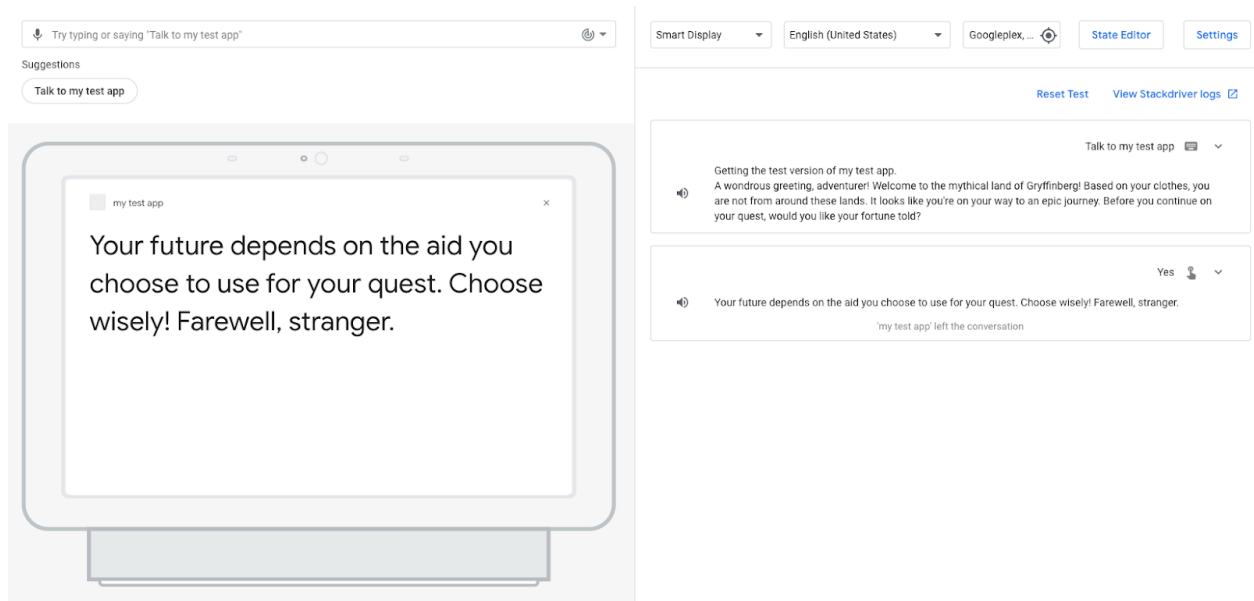
1. 在終端機執行下列指令：

```
gactions deploy preview
```

輸出內容應如下所示：

```
✓ Done. You can now test your changes in Simulator with this URL:
http://console.actions.google.com/project/{project-id}/simulator?disableAutoPreview
```

2. 複製提供的網址並貼到瀏覽器中。
3. 按一下「測試」即可前往模擬器。
4. 如要在模擬器中測試動作，請在「輸入內容」欄位中輸入 `Talk to my test app`，然後按 `Enter`。
5. 在「Input」欄位中輸入 `Yes`，然後按下 `Enter` 鍵。或者，按一下「是」建議方塊。



你的動作會回應使用者，並告知他們運勢取決於所選的協助。由於您為 `yes` 意圖設定 `End conversation` 轉換，因此 Action 會結束工作階段。

建立 `no` 意圖

現在您可以建立 `no` 意圖，讓「動作」在使用者不想聽取運勢時，瞭解並回應使用者。

如要建立這項意圖，請按照下列步驟操作：

1. 在終端機中，於 `intents` 目錄中建立名為 `no.yaml` 的新檔案：

```
touch custom/intents/no.yaml
```

2. 在文字編輯器中開啟 `no.yaml`。
3. 將下列訓練詞組貼到 `no.yaml` 檔案中：

no.yaml

```
trainingPhrases:
- nope
- I don't want
- "n"
- "no"
- nah
- no thanks
```

4. 儲存檔案。

將 no 意圖新增至 start 場景

現在，當使用者說出「否」或類似「否」的字詞 (例如「不要」)，動作就能理解。您需要將 no 自訂意圖新增至 Start 場景，因為使用者正在回應 Start 提示 (「繼續冒險前，要不要先算算命？」)。

如要為 Start 場景新增這項意圖，請按照下列步驟操作：

1. 在文字編輯器中開啟 `custom/scenes/Start.yaml`。
2. 在 `Start.yaml` 中，於 `yes` 處理常式下方新增下列 `no` 處理常式：

Start.yaml

```
- handler:
  staticPrompt:
    candidates:
      - promptResponse:
          firstSimple:
            variants:
              - speech: I understand, stranger. Best of luck on your quest! Farewell.
          intent: "no"
          transitionToScene: actions.scene.END_CONVERSATION
```

3. 儲存檔案。

在模擬工具中測試 no 意圖

此時，動作會瞭解使用者不想聽取運勢，並傳回適當的回應。

如要在模擬器中測試這項意圖，請按照下列步驟操作：

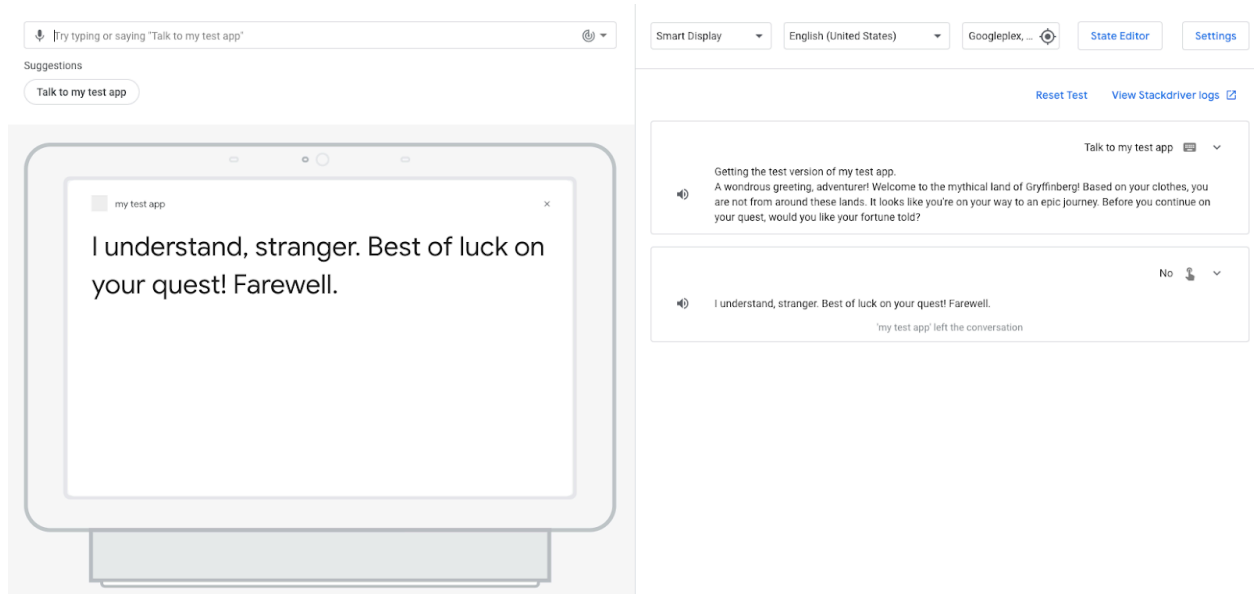
1. 在終端機執行下列指令：

```
gactions deploy preview
```

輸出內容應如下所示：

```
✓ Done. You can now test your changes in Simulator with this URL:
http://console.actions.google.com/project/{project-id}/simulator?disableAutoPreview
```

2. 複製提供的網址並貼到瀏覽器中。
3. 按一下「測試」即可前往模擬器。
4. 在「Input」欄位中輸入 `Talk to my test app`，然後按下 `Enter` 鍵。
5. 在「Input」欄位中輸入 `No`，然後按下 `Enter` 鍵。或者，按一下 `No` 建議方塊。



Action 不會為使用者提供運勢，而是祝他們旅途順利。由於您為 `no` 意圖設定 `End conversation` 轉換，因此 Action 會結束工作階段。

5. 實作執行要求

目前動作的回應是靜態的，也就是說，當含有提示的場景啟動時，動作每次都會傳送相同的提示。在本節中，您將實作「執行要求」，其中包含建構動態對話式回應的邏輯。

重要字詞：

- **執行：**包含動作邏輯的程式碼。Webhook 會根據 Actions 中發生的事件，觸發對執行要求的呼叫。

執行要求會判斷使用者是回訪還是新使用者，並修改回訪使用者的 Action 歡迎訊息。如果是回訪者，問候訊息會縮短，並感謝使用者回訪：「冒險家，歡迎回來！歡迎回到神秘的格里芬堡！」

在本程式碼研究室中，您將使用 *Cloud Functions 編輯器* 編輯及部署執行要求程式碼。

重要字詞：

- **Cloud Functions 編輯器：**Actions 專案的整合式編輯器，可用於透過 [Cloud Functions for Firebase](#) 編輯及部署執行要求程式碼。

您的動作可以觸發網路鉤子，在叫用期間或場景執行的特定部分發生事件時，通知您的完成情況。觸發 Webhook 時，動作會將要求連同處理事件的處理常式名稱，一併以 JSON 酬載的形式傳送至執行要求。這個處理常式會執行一些邏輯，並傳回相應的 JSON 回應。

建構執行要求

在本節中，您將修改完成動作，以便在回訪者和新使用者叫用動作時，產生不同的提示。

如要將這項邏輯新增至執行要求，請按照下列步驟操作：

1. 在終端機中，確認您位於專案的根目錄，然後建立新的 `webhooks` 目錄：

```
mkdir webhooks
```

2. 在 `webhooks` 目錄中建立名為 `ActionsOnGoogleFulfillment.yaml` 的新檔案：

```
touch webhooks/ActionsOnGoogleFulfillment.yaml
```

3. 在文字編輯器中開啟 `ActionsOnGoogleFulfillment.yaml`。
4. 將 `greeting` 處理常式和 `inlineCloudFunction` 內容新增至 `ActionsOnGoogleFulfillment.yaml` 檔案：

ActionsOnGoogleFulfillment.yaml

```
handlers:
- name: greeting
  inlineCloudFunction:
    executeFunction: ActionsOnGoogleFulfillment
```

`ActionsOnGoogleFulfillment.yaml` 檔案會定義 Webhook 處理常式 (例如 `greeting` 處理常式)，並指示動作將 Cloud Functions 做為 Webhook 端點。

5. 在 `webhooks` 目錄中建立新的 `ActionsOnGoogleFulfillment` 目錄：

```
mkdir webhooks/ActionsOnGoogleFulfillment
```

6. 在 `ActionsOnGoogleFulfillment` 目錄中建立名為 `index.js` 的新檔案：

```
touch webhooks/ActionsOnGoogleFulfillment/index.js
```

7. 在文字編輯器中開啟 `index.js`。

8. 在 `index.js` 加入以下程式碼：

index.js

```
const { conversation } = require('@assistant/conversation');
const functions = require('firebase-functions');

const app = conversation({debug: true});

app.handle('greeting', conv => {
  let message = 'A wondrous greeting, adventurer! Welcome back to the mythical land of Gryffinberg!';
  if (!conv.user.lastSeenTime) {
    message = 'Welcome to the mythical land of Gryffinberg! Based on your clothes, you are not from around these lands. It looks like you\'re on your way to an epic journey.';
  }
  conv.add(message);
});

exports.ActionsOnGoogleFulfillment = functions.https.onRequest(app);
```

這個程式碼會定義 `greeting` 處理常式，將適當的問候訊息傳送至使用者。

9. 儲存檔案。

10. 在 `ActionsOnGoogleFulfillment` 目錄中建立名為 `package.json` 的新檔案：

```
touch webhooks/ActionsOnGoogleFulfillment/package.json
```

`package.json` 檔案會指定 Webhook 的依附元件和其他中繼資料。

11. 在文字編輯器中開啟 `package.json`。

12. 從這個 [GitHub 存放區](#) 複製程式碼，然後貼到 `package.json` 檔案中。

13. 儲存檔案。

瞭解程式碼

執行要求會使用 [Node.js 適用的 Actions on Google 執行要求資料庫](#)，回應 Google 助理的 HTTP 要求。

在上述程式碼片段中，您定義了 `greeting` 處理常式，用於檢查使用者先前是否曾透過 `lastSeenTime` 屬性造訪動作。如果未定義 `lastSeenTime` 屬性，則使用者為新使用者，並會收到新使用者專屬的問候訊息。否則，訊息會確認使用者已返回，並產生修改後的問候語。

更新主要呼叫，觸發 Webhook

定義 `greeting` 函式後，您可以在主要叫用意圖中設定 `greeting` 事件處理常式，讓 Action 知道使用者叫用 Action 時要呼叫這個函式。

如要設定動作來呼叫新的 `greeting` 處理常式，請按照下列步驟操作：

1. 在文字編輯器中開啟 `custom/global/actions.intent.MAIN.yaml`。
2. 將 `actions.intent.MAIN.yaml` 中的程式碼替換成以下程式碼：

actions.intent.MAIN.yaml

```
handler:
  webhookHandler: greeting
transitionToScene: Start
```

3. 儲存檔案。

現在，當系統比對出主要叫用意圖時，就會呼叫 `greeting` 網頁服務掛鉤處理常式。

在模擬工具中測試更新後的主要叫用

如要在模擬器中測試動作，請按照下列步驟操作：

1. 在終端機執行下列指令：

由於這項指令會部署 Webhook 並安裝依附元件，因此可能需要較長時間。

```
gactions deploy preview
```

如果收到有關 Cloud Build API 的 403 錯誤，或是與帳單帳戶建立關聯時發生 403 錯誤，請參閱「[啟用、停用或變更專案的計費功能](#)」。如要使用 Cloud Functions 部署，您必須在 Google Cloud 中將帳單帳戶與專案建立關聯。關聯帳單帳戶後，請再次執行 `gactions deploy preview`。如要避免產生費用，請按照本程式碼研究室結尾「清除專案」一節中的步驟操作。

輸出內容應如下所示：

```
✓ Done. You can now test your changes in Simulator with this URL:
http://console.actions.google.com/project/{project-id}/simulator?disableAutoPreview
```

2. 複製提供的網址並貼到瀏覽器中。

3. 如要在模擬器中測試動作，請在「輸入內容」欄位中輸入 `Talk to my test app`，然後按 `Enter`。

由於您在本程式碼研究室中測試過動作，因此並非新使用者，會收到以下簡短的问候語：「冒險家，您好！歡迎回到神秘的格里芬堡！...」

6. 使用動作建構工具視覺化呈現動作

Actions SDK 可與整合至 Actions 管理中心的網頁式 IDE (稱為 [Actions Builder](#)) 互通。您可以使用 `gactions push` 指令，將本機檔案系統推送至控制台中的動作草稿。`gactions deploy preview` 只允許您在模擬器中測試動作，`gactions push` 則會將所有內容從本機檔案移至 Actions Builder。

Actions 控制台會以圖像化方式呈現動作的設定。在開發期間，以視覺化方式查看動作對應關係很有幫助，且不會影響用於測試的動作版本。

如果您與他人協作，執行 `gactions push` 會覆寫目前 Actions 專案的草稿。

如要推送 Actions 專案並在 Actions 控制台中查看，請按照下列步驟操作：

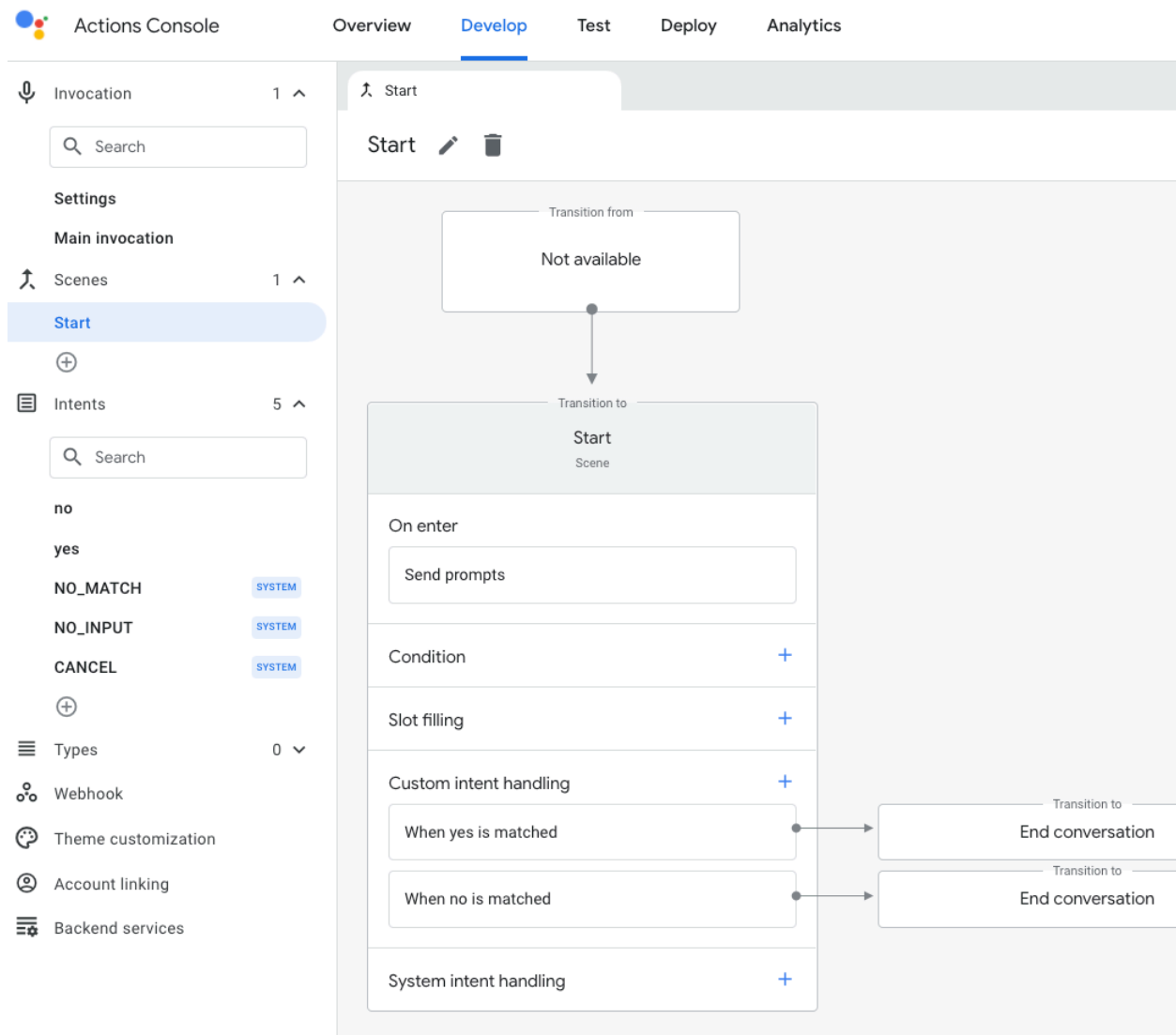
1. 在終端機中執行下列指令，將專案推送至 Actions 控制台：

```
gactions push
```

輸出內容應如下所示：

```
✓ Done. Files were pushed to Actions Console, and you can now view your project with
this URL: https://console.actions.google.com/project/{project-id}/overview. If you
want to test your changes, run "gactions deploy preview", or navigate to the Test
section in the Console.
```

2. 複製提供的網址並貼到瀏覽器中。
3. 在 [Actions 控制台](#) 中，按一下頂端導覽列中的「開發」。
4. 按一下「場景」旁的下拉式箭頭，然後按一下「開始」。您應該會看到 Action `start` 場景的視覺化呈現方式，如下方螢幕截圖所示：



清除專案所用資源 (建議)

注意：第 2 級程式碼研究室「[使用 Actions SDK 建構 Google 助理動作 \(第 2 級\)](#)」會以您在本程式碼研究室中建立的專案為基礎。如果您打算進行第 2 級程式碼研究室，請不要刪除專案，而是在第 2 級程式碼研究室結尾的「清除專案」一節中刪除專案。

為避免產生可能的費用，建議您移除不打算使用的專案。如要刪除您在本程式碼研究室中建立的專案，請按照下列步驟操作：

1. 如要刪除 Cloud 專案和資源，請完成「[關閉 \(刪除\) 專案](#)」一節列出的步驟。

請務必在「Google Cloud Platform 設定」頁面中，選取要刪除的正確專案。

2. 選用：如要立即從 Actions 管理中心移除專案，請完成「[刪除專案](#)」一節列出的步驟。如果未完成這個步驟，系統會在約 30 天後自動移除專案。

步驟 7 · 約 0 分鐘

7. 恭喜！

您已瞭解如何使用 Actions SDK 建構 Google 助理動作。

涵蓋範圍

- 如何在 Actions 控制台中設定 Actions 專案
- 如何使用 Actions SDK 在本機檔案系統中建構 Actions 專案
- 如何將提示新增至主要叫用，讓使用者與動作展開對話
- 如何使用場景、意圖、轉場效果、建議晶片和完成動作，建立對話式介面
- 如何使用 Actions 模擬器測試動作

瞭解詳情

如要進一步瞭解如何建構 Google 助理動作，請參閱下列資源：

- [開發 Google 助理動作的官方說明文件網站](#)
- [Actions on Google GitHub 頁面](#)，提供程式碼範例和程式庫
- 專為使用 Google 助理的開發人員開設的[官方 Reddit 社群](#)

在 Twitter 上追蹤 [@ActionsOnGoogle](#)，掌握最新公告，並使用 [#AoGDevs](#) 分享您建構的內容！

意見回饋問卷調查

離開前，請填寫簡短的[問卷調查](#)，分享你的體驗。
